

JsShaker

Dependency-Tracked Abstract Interpretation for
JavaScript Code Size Optimization

熊桐睿 许逸凡

指导老师：张昱 丁伯尧

项目时间线



NEW

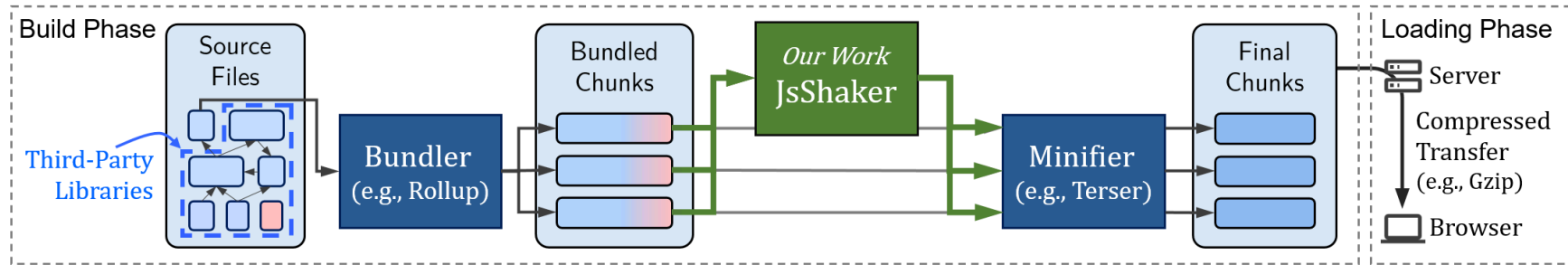
背景

- Octoverse 2025: TypeScript 和 JavaScript 成为 **top2 最广为使用的语言**
- 平均每个应用依赖的 npm 包数量高达 **350** 个，带来了大量未使用的代码
- 代码膨胀的后果：**加载变慢，用户体验下降 / 开销增大，Serverless 和 IoT 场景受限**
- Example: 无用代码的静态优化空间巨大：

```
1 function f(x) { if(x) return +x; else return input(); } → 1 function f() { return input(); }  
2 console.log(f("42"), f(null));                        2 console.log(42, f());
```

现有主流解决方案

组合 Bundler 和 Minifier 工具，优化代码体积：



Bundler

- 合并多文件，剪除未用模块
- 关键技术：可达性分析
- 代表工具：Rollup

Minifier

- 对语法做紧凑化重写
- 关键技术：多轮 AST 迭代变换
- 代表工具：Terser, Closure Compiler

现有技术路线

技术路线	代表工具	核心局限
可达性分析	Rollup, Webpack, Rolldown, Lacuna	仅识别模块/函数级死代码
多轮 AST 重写	Terser, esbuild, swc, Closure Compiler	依赖预定义模式，缺乏语义洞察
抽象解释	Prepack (已停止维护)	符号执行 + 堆序列化过于复杂，难以支持完整语义
运行时裁剪	Stubbifier, DepPrune, JSAnalyzer	依赖测试覆盖或脚本级模式，不安全且速度极慢

本质问题



思路

语言动态性 & 优化效果

→ 抽象解释

编译速度 & 抽象解释开销大

→ 单轮分析优化

~~DCE: 删除死代码~~

Tree-shaking: 保留活代码

→ 带有依赖追踪的抽象解释

挑战

```
1  const obj = {};  
2  document.onclick = function() { console.log(obj.foo); }; // 逃逸的回调函数  
3  obj[ThirdPartyLib.check() ? 'foo' : 'bar'] = 2; // ThirdPartyLib 对分析是不可见的  
4  Object.setPrototypeOf(obj, { foo: 3 }); // 动态原型修改: obj.foo 现在是 2 或 3
```

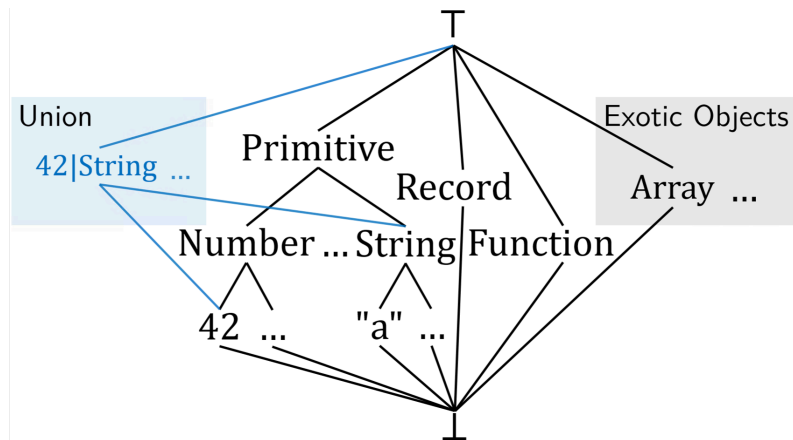
1. 动态控制流 (Line 2): 无法构建静态控制流图
2. 动态对象字段 (Line 3, 4): 难以实现字段敏感分析
3. 输入为部分程序 (Line 3): 必须对未知代码做保守假设
4. 源到源转换的约束: 不适合使用中间表示
5. 严格的构建时间要求: 不适合多次执行昂贵的抽象解释 (单轮分析优化)

核心设计: Dependency-Tracked Abstract Value

抽象值：对程序运行时真实数据的一种近似表示

程序运行时的具体值有无穷种可能，不可能枚举。

于是用“抽象值”来概括一类可能的具体值——它可以是“某个数字”“某个字符串”，甚至精确到“数字 42”。整个取值集合叫做“**抽象域**”：



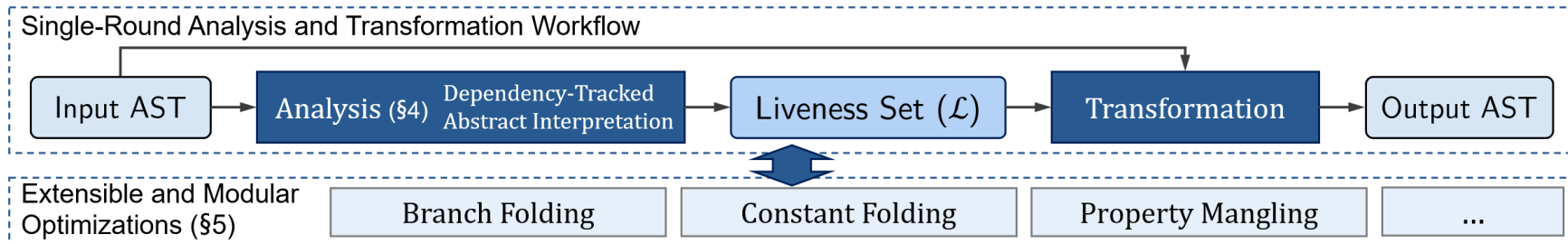
依赖追踪：本工作的核心机制

不仅计算一个变量“是什么”（抽象值），还维护一个集合，记录哪些代码节点（比如某行赋值、某个函数调用）参与生成了这个值。

当这个值最终产生副作用（比如打印输出）或从可分析代码逃逸时，系统就把集合里的代码节点标记为 **Live（不可删除）**。

基于此，优化器能在**单次分析**中精确判断哪些代码不可或缺、哪些可以安全删除，避免了传统方法需要多轮“分析—转换”的高昂开销。

整体架构



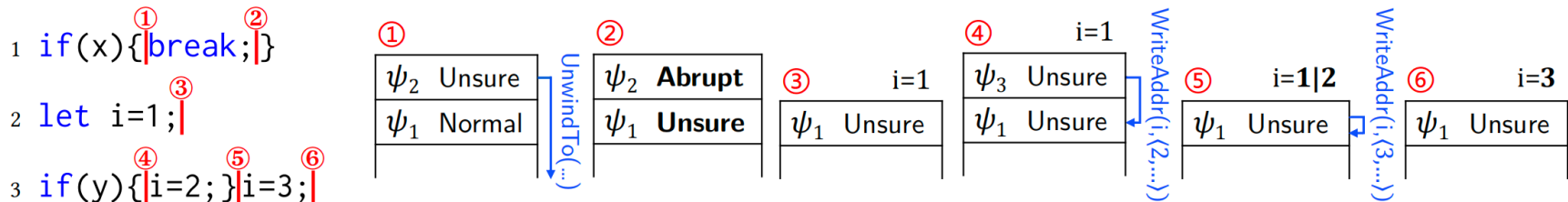
核心步骤 (实现了基础的死代码优化)

- 静态分析 (抽象解释)
with **Dependency-Tracked Abstract Value**
- 代码转换

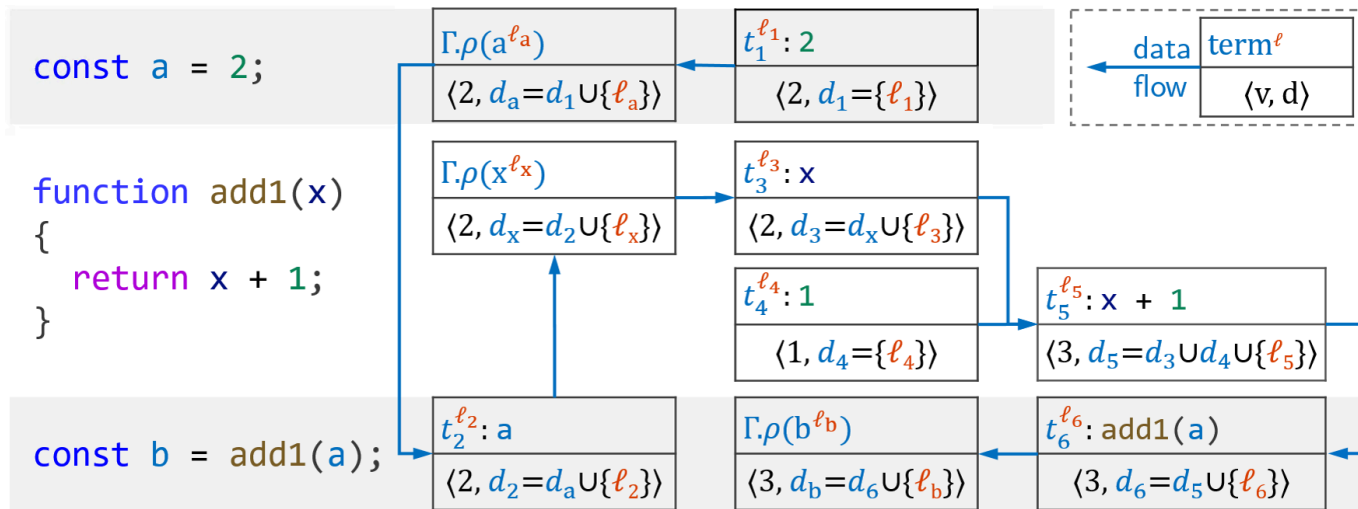
高级优化

- 分支折叠
- 常量折叠
- 属性名简化
- ... (可拓展)

控制流分析 – 控制状态、控制帧、控制栈 / 确定性与依赖传播/ 非顺序控制流



数据流分析 – 求值规则 / 对象语义 / 逃逸值



数据流分析规则

$$[op_1; op_2; \dots] \triangleright \llbracket t^\ell \rrbracket \Downarrow \langle v, d \rangle$$

在全局状态下，抽象解释在位置 ℓ 的表达式 t 的规则为

1. 执行操作 $op_1; op_2; \dots$ 2. 并得到抽象值 v 和依赖集合 d

Term t^ℓ	[Operational Steps $op_1; op_2; \dots$] $\triangleright$ Evaluation Result $\llbracket t^\ell \rrbracket \Downarrow e$
CONST: v	- $\llbracket t^\ell \rrbracket \Downarrow \langle v, \{\ell\} \rangle$
BINOP: $t_1 \odot t_2$	$\llbracket t_1 \rrbracket \Downarrow \langle v_1, d_1 \rangle; \llbracket t_2 \rrbracket \Downarrow \langle v_2, d_2 \rangle; \llbracket t^\ell \rrbracket \Downarrow \langle v_1 \odot v_2, d_1 \cup d_2 \cup \{\ell\} \rangle$
READ: x	$\langle v, d \rangle = x \in \text{dom}(\Gamma.\rho) ? \Gamma.\rho(x).e : \langle \top, \emptyset \rangle; \llbracket t^\ell \rrbracket \Downarrow \langle v, d \cup \{\ell\} \rangle$
WRITE: $x = t_1$	$\llbracket t_1 \rrbracket \Downarrow \langle v_1, d_1 \rangle; \text{if } (x \in \text{dom}(\Gamma.\rho)) \{ \Gamma.\text{WriteAddr}(\Gamma.\rho(x), \langle v_1, d_1 \cup \{\ell\} \rangle); \}$ $\text{else } \{ \Gamma.\text{SideEffect}(d_1 \cup \{\ell\}); \text{Escaped}(v_1); \} \llbracket t^\ell \rrbracket \Downarrow \langle v_1, d_1 \rangle$
ObjLit: $\{\pi_i[t_{ki}]: t_{vi}^{\ell_i}\}$ ($\pi ::= \text{get} \mid \text{set} \mid \epsilon$)	$r = \text{Record}(\text{Object.prototype}, \emptyset);$ foreach i do { $\llbracket t_{ki} \rrbracket \Downarrow \langle v_{ki}, d_{ki} \rangle; \llbracket t_{vi} \rrbracket \Downarrow \langle v_{vi}, d_{vi} \rangle; e_i = \langle v_{vi}, d_{ki} \cup d_{vi} \cup \{\ell\} \rangle;$ $r[v_{ki}] = \{\pi_i = \epsilon ? \text{Data}(\langle e_i, \Gamma.\psi \rangle) : \text{Accessor}(\dots)\}; \} \llbracket t^\ell \rrbracket \Downarrow \langle r, \{\ell\} \rangle$
PROPGET*: $t_1[t_2]$	$\llbracket t_1 \rrbracket \Downarrow \langle v_1, d_1 \rangle; \llbracket t_2 \rrbracket \Downarrow \langle v_2, d_2 \rangle; d = d_1 \cup d_2 \cup \{\ell\}; \Theta = \text{ReadDescriptors}(v_1, v_2);$ $\langle v_d, d_d \rangle = \sqcup \{a.e \mid \text{Data}(a) \in \Theta\}; e_d = \langle v_d, d_d \cup d \rangle;$ $\Gamma.\text{Push}(\text{PropAccess}, \Theta > 1 ? \text{Unsure} : \text{Normal}, d);$ $e_a = \sqcup \{\Gamma.\text{Call}(e_{\text{get}}, [], \emptyset) \mid \text{Accessor}(e_{\text{get}}, e_{\text{set}}) \in \Theta\}; \Gamma.\text{Pop}(); \llbracket t^\ell \rrbracket \Downarrow e_d \sqcup e_a$
PROPSET*: $t_1[t_2] = t_3$	$\llbracket t_1 \rrbracket \Downarrow \langle v_1, d_1 \rangle; \llbracket t_2 \rrbracket \Downarrow \langle v_2, d_2 \rangle; \llbracket t_3 \rrbracket \Downarrow e_3; \Theta = \text{WriteDescriptors}(v_1, v_2);$ $\Gamma.\text{Push}(\text{PropAccess}, \Theta > 1 ? \text{Unsure} : \text{Normal}, d_1 \cup d_2 \cup \{\ell\});$ foreach $\text{Data}(a) \in \Theta$ do { $\Gamma.\text{WriteAddr}(a, e_3); \}$ foreach $\text{Accessor}(e_{\text{get}}, e_{\text{set}}) \in \Theta$ do { $\Gamma.\text{Call}(e_{\text{set}}, [e_3], \emptyset); \} \Gamma.\text{Pop}(); \llbracket t^\ell \rrbracket \Downarrow e_3$

* For brevity, some details like null checking are not formalized.

过程间分析

- 必要性：JavaScript 函数是**一等公民**；**函数特化**是重要的优化机会。
- 设计：**上下文敏感**的过程间分析；递归/逃逸的穷举分析机制。

函数摘要

- 必要性：如果每次遇到函数调用都把函数体重新展开分析一遍，那么会让分析时间呈**指数级增长**。
- 设计：函数摘要本质上是一份“**复用缓存**”：它把函数针对不同输入的行为（返回值、修改了哪些外部变量、有没有副作用）提炼成模板。下次再遇到兼容的调用时直接套用这份缓存，而不必重新遍历函数体。
- 难点：和依赖分析兼容；平衡摘要的精确度和复用率。
- 效果：将优化速度提升了平均 1.58，最高 23.28 倍。

高级优化：分支折叠

```
function assertPositive(x) {  
-   if (x > 0) {  
       console.log('positive', x);  
-   } else {  
-       throw new Error('non-positive');  
-   }  
}  
assertPositive(1);
```

Formal Rules

Evaluation	$\left[\begin{array}{l} \llbracket t_1 \rrbracket \Downarrow \langle v, d \rangle; \mathcal{B} = \mathcal{B} \cup \{ \langle \ell, \Gamma.c, v, d \rangle \}; \\ \sigma = (v = \top_{\text{Bool}}) ? \text{Unsure} : \text{Normal}; \\ \text{if } (\text{True} \sqsubseteq v) \{ \Gamma.\text{Push}(\text{Branch}, \sigma, \{ \ell \}); \llbracket t_2 \rrbracket \Downarrow \text{Ok}; \Gamma.\text{Pop}(); \} \\ \text{if } (\text{False} \sqsubseteq v) \{ \Gamma.\text{Push}(\text{Branch}, \sigma, \{ \ell \}); \llbracket t_3 \rrbracket \Downarrow \text{Ok}; \Gamma.\text{Pop}(); \} \end{array} \right] \triangleright \llbracket \text{if}^\ell(t_1) \ t_2 \ \text{else} \ t_3 \rrbracket \Downarrow \text{Ok}$
------------	--

Inclusion	$\frac{\langle \ell, c_1, v_1, d_1 \rangle \in \mathcal{B} \quad \langle \ell, c_2, v_2, d_2 \rangle \in \mathcal{B} \quad \{ \ell, c_1, c_2 \} \subseteq \mathcal{L} \quad \text{True} \sqsubseteq v_1 \quad \text{False} \sqsubseteq v_2}{\ell \in \mathcal{N} \quad d_1 \cup d_2 \subseteq \mathcal{L}}$
-----------	---

Transformation	$\text{if}^\ell(\widehat{t_1} \ \widehat{t_2} \ \text{else} \ t_3) = \begin{cases} \text{if } (\widehat{t_1}) \ \widehat{t_2} \ \text{else} \ \widehat{t_3}, & \text{if } \ell \in \mathcal{N} \\ \widehat{t_1}; \ \widehat{t_2}; \ \widehat{t_3}, & \text{otherwise} \end{cases}$
----------------	--

高级优化：常量折叠

```
function compute(x, y) {  
-   if (x === 0) return -1;  
-   return Math.random() * x + y;  
+   return Math.random() * 1 + 2;  
}  
- console.log(compute(0), compute(1, 2));  
+ console.log(-1, compute());
```

Formal Rules

Evaluation	$\llbracket t \rrbracket^{\text{base}} \Downarrow \langle v, d \rangle \triangleright \llbracket t^\ell \rrbracket \Downarrow \langle v, \{\text{Fold}(\ell, v, d)\} \rangle$
Resolution	$\mathcal{K}(\ell) \triangleq \bigsqcup \{v \mid \exists d. \text{Fold}(\ell, v, d) \in \mathcal{L}\}$
Inclusion	$\frac{\mathcal{K}(\ell) \text{ is not a literal} \quad \text{Fold}(\ell, v, d) \in \mathcal{L}}{d \subseteq \mathcal{L}}$
Transformation	$\widehat{t}^\ell = \begin{cases} (\widehat{t}^{\text{base}}, \mathcal{K}(\ell)), & \text{if } \mathcal{K}(\ell) \text{ is a literal} \\ \widehat{t}^{\text{base}}, & \text{otherwise} \end{cases}$

高级优化：属性名简化

据估计，属性名约占打包后代码体积的 **30%**，具有巨大的优化空间，而现有的优化工具不能实现安全优化。

```
function getInfo(id) {  
-   return { age: ..., name: ... };  
+   return { a: ..., b: ... };  
}  
const user = getInfo(42);  
- console.log(user.age, user.name);  
+ console.log(user.a, user.b);
```

Formal Rules

Mangling Candidate	$\mathcal{V} \ni \text{MglStr}(s, \ell) \quad [] \triangleright \llbracket s^\ell \rrbracket \Downarrow \langle \text{MglStr}(s, \ell), \{\ell\} \rangle$
Mangling Constraint	$\delta ::= \dots \mid \ell_1 \sim \ell_2 \mid \ell_1 \not\sim \ell_2 \mid \ell \equiv s$
Comparison Evaluation	$\left[\llbracket t_1 \rrbracket \Downarrow \langle v_1, d_1 \rangle; \llbracket t_2 \rrbracket \Downarrow \langle v_2, d_2 \rangle \right] \triangleright \llbracket (t_1 === t_2)^\ell \rrbracket \Downarrow \langle v_1 \stackrel{?}{=} v_2, d_1 \cup d_2 \cup \text{Compare}(v_1, v_2) \cup \{\ell\} \rangle$
Resolution	Resolve $\mathcal{M}$ via a simple greedy solver
Transformation	$\hat{t}^\ell = \begin{cases} (\hat{t}^{\text{base}}, \mathcal{M}(\ell')), & \text{if } \mathcal{K}(\ell) = \text{MglStr}(s, \ell') \\ \hat{t}^{\text{CF}}, & \text{otherwise} \end{cases}$

Evaluation: Dataset

13 个由真实 JavaScript 项目构建的测试用例：

Category	Project (Target)	kLOC	Size (KB)	#File	#Pkg	Description	Stars	Date
Data Parsing	js-yaml (load)	2.8	105	2	1	YAML parser and serializer	6,557	2025-11
File System	glob (globSync)	5.0	245	19	7	File path pattern matching utility	8,722	2026-02
Functional	remeda (pipe,range,filter)	1.3	39	62	1	TypeScript-first functional utils	5,317	2026-02
Icons	react-icons (FaBeer)	6.9	1,356	8	2	SVG icon components for React	12,519	2026-03
Monitoring	sentry (init)	32.0	1,520	261	6	Error tracking and monitoring SDK	8,602	2026-03
Networking	novnc (UI.start)	13.6	650	58	1	VNC client built with HTML5	13,541	2026-01
Presentation	slidev-demo (frontend)	227.1	8,291	412	54	Slides framework for developers	44,850	2026-03
Reactivity	rxjs (range,map,filter)	8.5	332	201	2	Observable-based reactivity library	31,655	2025-02
UI Library	antd (DatePicker)	120.3	4,427	1,422	67	Enterprise-grade React.js UI	97,696	2026-03
UI Library	material-ui (Paper)	39.4	1,758	487	26	Material Design for React.js	98,003	2022-04
UI Library	vuetify (VApp,VBtn)	49.3	1,677	434	6	Component framework for Vue.js	40,998	2026-03
Utilities	lodash-es (flowRight,range,filter)	9.3	613	639	1	ESM build of the Lodash utilities	61,578	2026-01
Visualization	d3 (select,scaleLinear)	19.9	680	555	34	Data-driven DOM manipulation	112,538	2024-03
Total (deduped)		533.9	21,645	4,530	195			

Evaluation: Effectiveness

在所有测试用例上取得最优，额外删除平均 27.6%，最高可达 66.7%的代码体积：

Program	Baseline (KB)	JsShaker (React-aware)	CC _{Simp.}	CC _{Adv.}	LacunaO ₂	LacunaO ₃
antd	165.20	4.5% (18.7%)	0.1%	5.6%	<i>Syntax Err.</i>	90.0%
d3	15.55	52.8%	0.1%	14.1%	<i>Syntax Err.</i>	42.5%
glob	25.67	21.7%	<i>Unsupported</i>	<i>Unsupported</i>	<i>Syntax Err.</i>	<i>Syntax Err.</i>
js-yaml	12.53	27.0%	24.9%	26.5%	38.6%	38.1%
lodash-es	6.37	53.8%	28.6%	30.4%	2.4%	2.3%
material-ui	69.56	38.9% (40.5%)	-0.2%	5.4%	33.3%	33.3%
novnc	60.09	4.2%	-0.8%	8.8%	83.4%	<i>Syntax Err.</i>
react-icons	3.66	66.7% (74.9%)	0.1%	15.0%	33.1%	-181.6%
remeda	0.92	9.6%	2.0%	6.5%	0.4%	0.5%
rxjs	2.64	16.1%	1.9%	10.8%	88.9%	<i>Syntax Err.</i>
sentry	24.92	4.9%	2.9%	12.4%	<i>Syntax Err.</i>	<i>Syntax Err.</i>
slidev-demo	971.62	6.5%	1.4%	6.9%	OOM	OOM
vuetify	43.78	3.7%	-0.1%	8.6%	<i>Syntax Err.</i>	<i>Syntax Err.</i>
Geomean		27.6% (30.2%)	5.6%	16.0%	13.4%	N/A

CC_{Simp.} and CC_{Adv.} denote Google Closure Compiler v20260128 in Simple/Advanced modes.

Evaluation: Ablation Study

左：分支折叠、常量折叠和属性名简化这三个高级优化分别贡献了平均 12.0%, 12.7% 和 3% 的体积缩减。

右：JsShaker 安全的属性名简化可以识别并优化平均约 13.95% 的可优化属性名。

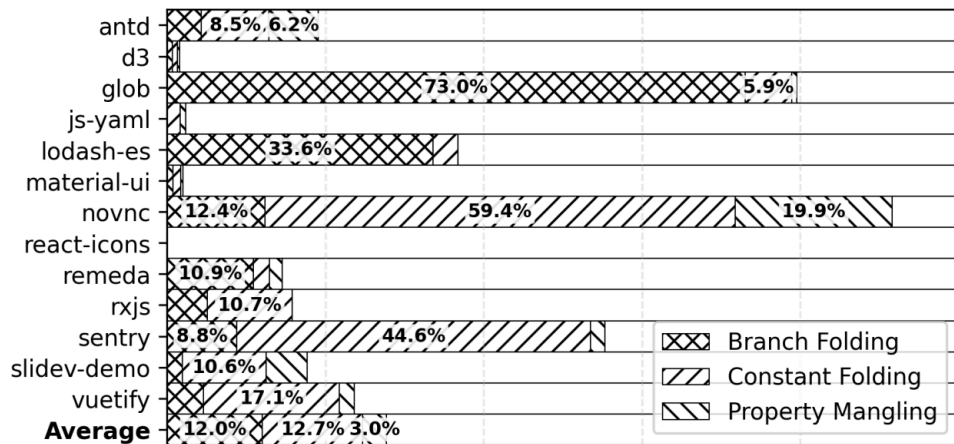


Table 6. Comparison of property mangling rates between JsShaker and Closure Compiler.

Program	Total	CC _{Adv.} (%)	JsShaker (%)	JsShaker/CC _{Adv.} (%)
anttd	19,158	41.38	4.31	10.42
d3	1,040	35.79	7.02	19.61
js-yaml	1,031	19.21	3.43	17.84
lodash-es	251	13.00	3.23	24.82
material-ui	6,255	47.65	3.11	6.52
novnc	8,354	61.08	12.42	20.33
react-icons	201	47.92	15.73	32.82
remeda	80	42.86	4.00	9.33
rxjs	301	40.51	0.00	0.00
sentry	2,549	44.23	3.79	8.57
vuetify	4,590	39.32	3.37	8.58
Average		39.36	5.49	13.95

Fig. 9. Ablation study of size reduction by JsShaker’s optimization strategies. Foundational tree-shaking (white segments) establishes the foundation, while advanced optimizations (hatched segments) vary across programs.

Evaluation: Performance

左：JsShaker 平均引入了 14% 的额外构建时间；右：MaxRecDepth=2 时较好地平衡了分析速度和优化效果。

Table 7. Build time overhead. Baseline (R+T:Rollup+Terser) execution times are in milliseconds, while other results are presented as multipliers (×) of the baseline. Rolldown (RD) serves as a Rust-based reference. Best results are bolded; “—” denotes build-time failures.

Project	With Baseline		Optimizer (×)				
	R+T (ms)	RD (×)	JsShaker	CC _{Simp.}	CC _{Adv.}	LacunaO ₂	LacunaO ₃
antd	2952	0.03	1.27	1.41	1.76	17.5	17.5
d3	321	0.05	1.01	2.07	3.29	13.2	13.1
glob	145	0.09	1.05	—	—	30.0	29.6
js-yaml	70	0.11	1.98	5.96	12.2	54.8	55.3
lodash-es	316	0.04	1.00	2.02	3.10	12.6	12.7
material-ui	921	0.04	1.05	1.65	2.27	11.8	11.9
novnc	353	0.07	1.11	2.40	4.30	14.1	14.0
react-icons	145	0.15	1.02	4.13	7.30	41.0	40.9
remeda	29	0.10	1.01	9.66	23.3	124	125
rxjs	150	0.03	1.00	2.67	5.46	25.2	25.5
sentry	406	0.04	1.02	1.82	3.34	11.3	11.3
slidev-demo	13075	—*	1.54	1.41	—	—	—
vuetify	801	0.06	1.06	1.52	2.20	10.4	10.5
Geomean	—	0.06	1.14	2.49	4.48	21.9	21.9

* slidev-demo is incompatible with Rolldown.

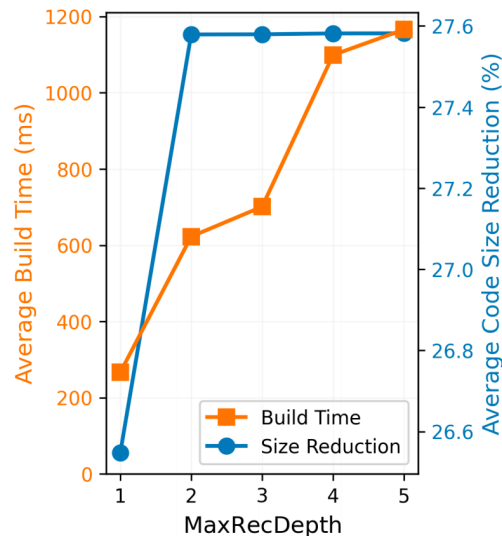


Fig. 10. Impact of MaxRecDepth on build time and code size. Beyond depth 2, build time increases with negligible size reduction.

Evaluation: Function Summary Effectiveness

函数摘要技术的引入，将优化速度提升了平均 1.58，最高 23.28 倍：

Project	#FunDef	#AbsInst	#Calls	Retrieval (%)			Generation (%)			Time (ms)		Speedup (×)
				ArgMiss	EnvMiss	Hit	PropRW	ObjRet	Updated	w/o Sum.	w/ Sum.	
antd	3,294	15,390	193,757	61.98	0.06	37.96	60.35	27.63	12.01	18725.03	804.41	23.28
d3	372	410	603	57.55	3.15	39.30	39.07	14.75	46.17	4.76	3.53	1.35
glob	525	619	1,129	77.41	1.51	21.08	41.41	3.14	55.44	8.00	7.79	1.03
js-yaml	144	183	26,697	50.56	0.00	49.44	62.75	36.14	1.11	79.54	90.69	0.88
lodash-es	154	155	202	68.81	0.00	31.19	25.90	9.35	64.75	1.61	1.46	1.10
material-ui	789	933	3,317	51.46	0.84	47.69	24.21	18.50	57.29	94.55	55.69	1.70
novnc	699	776	9,092	61.00	0.02	38.98	74.32	2.47	23.22	76.05	44.17	1.72
react-icons	1,625	1,629	35	100.00	0.00	0.00	68.57	25.71	5.71	3.52	3.54	0.99
remeda	23	27	36	100.00	0.00	0.00	44.44	5.56	50.00	0.37	0.37	0.99
rxjs	92	97	147	81.63	0.68	17.69	24.79	10.74	64.46	0.71	0.73	0.98
sentry	639	920	2,365	67.23	0.68	32.09	49.44	7.78	42.78	15.76	9.06	1.74
slidev-demo	17,153	151,659	1,264,575	73.74	0.15	26.11	67.49	14.51	17.99	10046.96	7045.21	1.43
vuetify	1,000	5,785	19,545	74.80	0.46	24.74	45.11	4.47	50.42	81.05	46.19	1.75
Average				71.71	0.14	28.15	66.24	16.01	17.74	2241.38	624.07	1.58

谢谢大家！

项目仓库: github.com/kermanx/tree-shaker

幻灯片: kermanx.com/jsshaker-ustc